Step 1 – Query Pair (Q, Q'')

Rewriter(Q, gemini-3.1-pro, Stats-CEB)

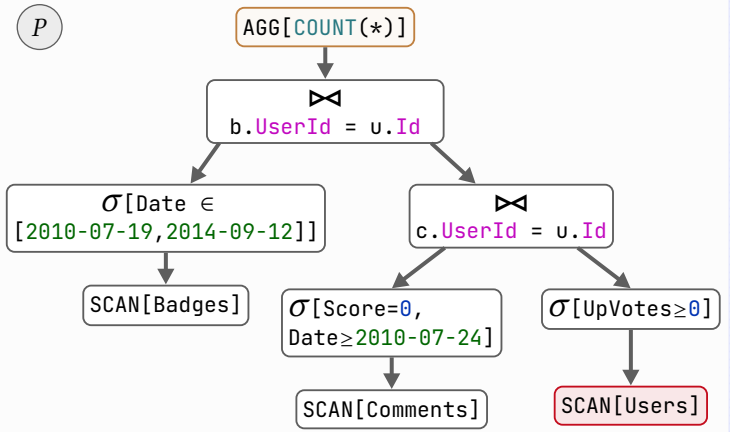
Original Query (Q)

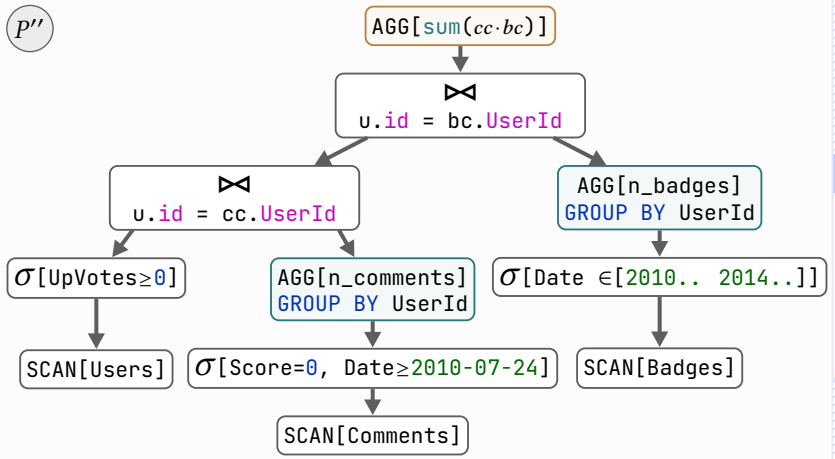
```
SELECT COUNT(*)
FROM Comments c, Badges b, Users u
WHERE c.UserId = u.Id
  AND b.UserId = u.Id
  AND c.Score = 0
  AND c.CreationDate >= '2010-07-24 06:46:49'
  AND b.Date >= '2010-07-19 20:34:06'
  AND b.Date <= '2014-09-12 15:11:36'
  AND u.UpVotes >= 0;
```

Rewritten Query (Q'')

```
WITH filtered_users AS (
  SELECT id FROM Users WHERE UpVotes >= 0),
comment_counts AS (
  SELECT UserId, count(*) AS n_comments
  FROM Comments
  WHERE Score = 0 AND CreationDate >= '2010-07-24 06:46:49' GROUP BY UserId),
badge_counts AS (
  SELECT UserId, count(*) AS n_badges
  FROM Badges WHERE Date BETWEEN '2010-07-19 20:34:06' AND '2014-09-12 15:11:36'
  GROUP BY UserId)
SELECT sum(cc.n_comments * bc.n_badges)
FROM filtered_users AS u
  JOIN comment_counts AS cc ON u.id = cc.UserId
  JOIN badge_counts AS bc ON u.id = bc.UserId;
```

Step 2 – Logical IR Trees P and P''





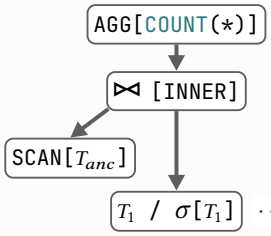
Anchor | Changed | Added | Unchanged

Step 3 – Structural Diff Δ

Pri.	Rule
1	JoinDecomposeToPreaggregatedCount
2	FilterIntoJoin
3	JoinReorder

Step 4 – Generalized Pattern & Keys

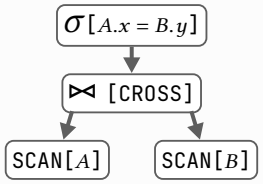
JoinDecomposeToPreaggregatedCount



Fragment Keys:
- $\mathcal{K}_1$ AGG(FILTER)
- $\mathcal{K}_2$ AGG(FILTER(JOIN[INNER]))
- $\mathcal{K}_3$ AGG(JOIN[INNER])
- $\mathcal{K}_4$ AGG(FILTER(JOIN[INNER](JOIN,SCAN)))
- $\mathcal{K}_5$ AGG(JOIN[INNER](JOIN,SCAN))

COUNT(*) over star join; $T_{anc}.id = T_i.fk$ for all i; no GROUP BY

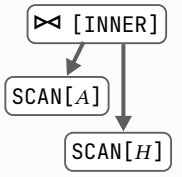
FilterIntoJoin



Fragment Keys:
- $\mathcal{K}_1$ FILTER(JOIN[CROSS])
- $\mathcal{K}_2$ FILTER(JOIN[INNER])

FILTER over CROSS/INNER JOIN; equality A.x=B.y spans both join inputs

JoinReorder



Fragment Keys:
- $\mathcal{K}_1$ JOIN[INNER]
- $\mathcal{K}_2$ JOIN[INNER](JOIN,SCAN)
- $\mathcal{K}_3$ JOIN[INNER](JOIN[INNER], SCAN)
- $\mathcal{K}_4$ JOIN[INNER](SCAN,SCAN)

INNER JOIN: hub H is not the leftmost (leading) scan in the join tree